# TP2 – Transferência de Arquivos Peer-to-Peer

Sistemas Distribuídos – CEFET-MG – Profa. Michelle Hanne – 2026/2
Parte 1: roteiro da apresentação · Parte 2: explicação do código (onde, como e por quê)

> **O que o enunciado exige (resumo):** peers simétricos (servidor + cliente no mesmo processo), seeder que fragmenta o arquivo em blocos fixos, leecher que conecta a todos os vizinhos e pede blocos até completar, bloco recebido e validado passa a ser servido (seeder em cascata), vizinhos pré-definidos (tracker opcional), sockets com concorrência, formato de metadados + protocolo próprio, registro de blocos, remontagem com verificação de integridade, testes da Tabela 1 (2 e 4 peers; bloco 1 KB e 4 KB; arquivos A = 10/20 KB, B = 1/5 MB, C = 10/20 MB) e relatório de até 5 páginas com a URL do código.

## PARTE 1 – Roteiro da apresentação (~10 min + demo)

| Tempo | Seção | O que dizer / mostrar |
|---|---|---|
| 0:00 – 1:00 | 1. Abertura | Objetivo: P2P elementar inspirado no BitTorrent; cada processo é servidor e cliente ao mesmo tempo; arquivo dividido em blocos e trocado direto entre os pares, sem servidor central. Implementação em **C** (sockets POSIX, pthreads, SHA-256 escrito do zero). Testes no Linux (WSL2), todos os peers em localhost. |
| 1:00 – 3:00 | 2. Arquitetura | Uma thread de `accept` (servidor) + uma thread de cliente por vizinho, no mesmo processo (`peer.c`). Lista de vizinhos estática (`--neighbors`), sem tracker. Controle de blocos por vetores `have[]` e `inflight[]` sob mutex. Seleção **rarest-first**. Escrita em `.part` e renomeação só após validar o SHA-256 do arquivo inteiro. |
| 3:00 – 4:30 | 3. Serialização e protocolo | Metadados `.meta` em texto (nome, tamanho, bloco, nº de blocos, SHA-256 do arquivo e de cada bloco). Mensagem TCP: `[tipo 1B][tamanho 4B big-endian][payload]` com 7 tipos: HELLO, GET_BITFIELD, BITFIELD, GET_BLOCK, BLOCK, NO_BLOCK, BYE. |
| 4:30 – 5:30 | 4. Integridade | Hash por bloco (bloco ruim é descartado e re-solicitado) + hash do arquivo final. Os scripts de teste ainda comparam com o `sha256sum` do sistema, que é independente do SHA-256 próprio. |
| 5:30 – 6:30 | 5. Metodologia | Matriz da Tabela 1: 36 cenários (2 peers mesh; 4 peers mesh e line), 5 rodadas cada, média e desvio padrão (o WSL2 tem jitter). Cada teste: preparar arquivo aleatório + meta → subir peers → logs → verificar SHA-256, tamanho e origem dos blocos. |
| 6:30 – 8:30 | 6. Demonstração | Rodar um cenário e mostrar nos logs: `conectado ao vizinho`, `DOWNLOAD COMPLETO … sha256=OK`, `origem … -> N blocos (x%)` e `agora atuando como SEEDER`. Comandos na seção abaixo. |
| 8:30 – 10:30 | 7. Resultados | Tabela e gráficos abaixo. Mensagem central: o tempo é proporcional ao **número de blocos** (≈ 0,6–0,8 ms por bloco com 2 peers), por isso 4 KB é ~3–4× mais rápido que 1 KB. |
| 10:30 – 11:30 | 8. Limitações e conclusão | Pull-based (sem mensagem HAVE), um pedido por vez por conexão, mutex global. Outliers vistos só no 1º benchmark. Todos os requisitos atendidos; integridade OK nos 36 cenários × 5 rodadas (2 benchmarks completos). **URL do repositório: <inserir>**. |

## Comandos da demonstração (no WSL)

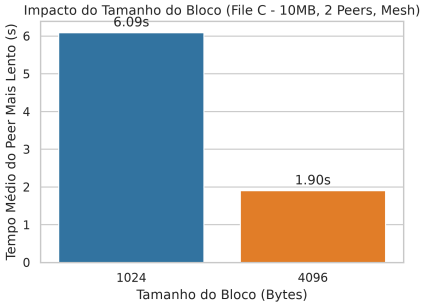
```
cd "/mnt/d/CEFET/Turmas/2026.2/sistemas distribuidos/sd_tp2"
make clean && make
bash scripts/run_test.sh 4 4096 5242880 mesh FileB     # 4 peers, bloco 4 KB, 5 MB
cat logs/FileB_p4_b4096_f5242880_mesh/B.log | tail -15  # origem dos blocos + SEEDER
column -s, -t results/results.csv | tail -2
```

## Resultados (2º benchmark: média de 5 rodadas; tempo do peer mais lento, em segundos)

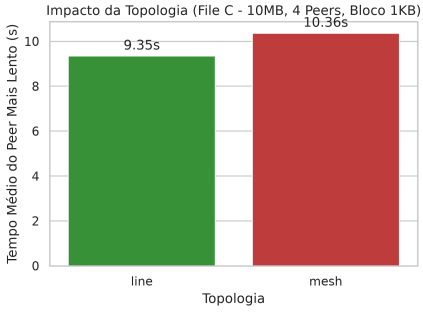
> **Atualização:** o benchmark foi refeito (180 execuções, 0 falhas de integridade/tamanho). O código-fonte em `src/` não mudou; mudaram só os scripts (geram PNG das tabelas). Os valores abaixo substituem os do 1º benchmark, que tinha outliers de até 105 s e desvios enormes na topologia line.

| Cenário (File C) | Bloco 1 KB | Bloco 4 KB | Razão |
|---|---|---|---|
| 10 MB, 2 peers, mesh | 6,09 ± 0,89 | 1,90 ± 0,25 | 3,2× |
| 20 MB, 2 peers, mesh | 11,83 ± 1,07 | 4,04 ± 0,54 | 2,9× |
| 10 MB, 4 peers, mesh | 10,36 ± 0,62 | 2,77 ± 0,43 | 3,7× |
| 20 MB, 4 peers, mesh | 20,29 ± 1,16 | 5,60 ± 0,23 | 3,6× |
| 10 MB, 4 peers, line | 9,35 ± 1,02 | 3,12 ± 0,38 | 3,0× |
| 20 MB, 4 peers, line | 17,47 ± 1,50 | 5,55 ± 0,92 | 3,1× |

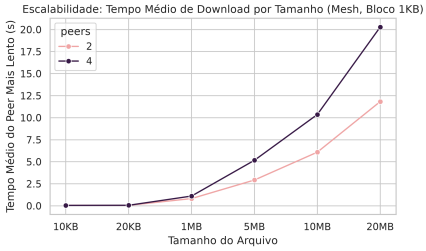
Todos os 36 cenários × 5 rodadas terminaram com **integridade (SHA-256) e tamanho OK**. Arquivos pequenos (File A) levam 0,05–0,3 s no peer; o "~1,0–1,2 s" de `tempo_total_s` é overhead do script de teste (`sleep 0.5` para o seeder subir + amostragem a cada 0,2 s).




Impacto do tamanho do bloco




Mesh × line (10 MB, 1 KB)




Escalabilidade (mesh, 1 KB)

Observação: o script `plot_results.py` desenha barras de erro (`errorbar='sd'`), mas como o CSV de médias tem uma linha por grupo, as barras não aparecem. O desvio padrão está na tabela acima.

## Como interpretar (para falar na apresentação)

- **Tempo ≈ nº de blocos × latência por bloco.** File C 20 MB, 2 peers: 1 KB = 20 480 blocos em 11,8 s ≈ 0,58 ms/bloco; 4 KB = 5 120 blocos em 4,04 s ≈ 0,79 ms/bloco. A latência por bloco varia pouco (com 4 KB cada bloco carrega mais bytes, então custa um pouco mais); o que muda muito é quantos blocos existem. Isso decorre de cada conexão fazer *pedido → resposta* um bloco por vez (sem pipelining). Por isso a razão 1 KB / 4 KB fica entre 2,9× e 3,7×, perto de 4× (a razão do número de blocos), mas não igual.
- **Mais peers deixou mais lento** (2 → 4 peers, mesh: 11,8 → 20,3 s com 1 KB; 4,0 → 5,6 s com 4 KB). Os leechers pegaram ~95–97% dos blocos do seeder (ex.: B recebeu 96,7% de A), então o seeder serviu ~3× mais dados; tudo roda na mesma máquina, disputando CPU e o mutex global. O efeito é menor com 4 KB (menos blocos, menos disputa).
- **Por que os leechers servem tão pouco?** O protocolo não tem mensagem HAVE: um peer só descobre os blocos novos de um vizinho ao reler o bitfield (a cada 256 blocos ou quando fica sem o que pedir). A

cascata funciona (`agora atuando como SEEDER`), mas a troca entre leechers é pequena.

- **Line × mesh (4 peers):** sem vencedor consistente. Com 1 KB a line foi cerca de 10–14% mais rápida (File C: 9,35 × 10,36 s e 17,47 × 20,29 s); com 4 KB empatou ou ficou até ~15–20% atrás (File C 10 MB: 3,12 × 2,77 s; File B 5 MB: 1,67 × 1,41 s). Explicação plausível para o ganho em 1 KB: na line o seeder atende só 1 vizinho, o que reduz a disputa por ele. As diferenças são da ordem de 1–2 desvios padrão; dizer "comparável", sem exagerar.
- **Estabilidade:** no 2º benchmark os desvios padrão ficaram em ~5–15% da média em File B/C e não houve outliers. No 1º benchmark havia execuções com até 104,7 s (FileA 10 KB / 4 KB / line), um múltiplo de 15 s (`RECV_TIMEOUT_S`, `peer.c:31`) que sugere espera por timeout de `recv`. Como o código de `src/` é o mesmo nos dois benchmarks, a causa provável é intermitente/ambiental (WSL2, carga da máquina), **mas não foi confirmada**. Declarar como limitação conhecida.

# PARTE 2 – Explicação do código

## 2.1 Mapa dos arquivos (onde está cada coisa)

| Arquivo | Linhas | Responsabilidade |
|---|---|---|
| `src/peer.c` | 426 | O peer inteiro: estado, log, armazenamento, servidor, cliente, rarest-first, remontagem e `main`. |
| `src/mkmeta.c` | 56 | Ferramenta que lê o arquivo original e gera o `.meta` com os SHA-256. |
| `src/meta.c` / `include/meta.h` | 74 | Ler/gravar/validar o `.meta`; `meta_block_len` (tamanho real de cada bloco). |
| `src/protocol.c` / `include/protocol.h` | 66 | Enquadramento das mensagens TCP: `send_all`, `recv_all`, `msg_send`, `msg_recv`. |
| `src/sha256.c` | 102 | SHA-256 implementado do zero (sem OpenSSL): `init/update/final`, `sha256_buf`, `sha256_file`. |
| `src/util.c` | 35 | `now_s()` (relógio monotônico), `hex_encode` / `hex_decode`. |
| `scripts/run_test.sh` | – | Executa UM cenário: prepara dados, sobe peers, monitora, verifica e grava no CSV. |
| `scripts/run_all.sh`, `run_benchmark.sh` | – | Matriz da Tabela 1; benchmark repetido N vezes. |
| `tools/*.py` | – | `average_results.py` (média/desvio) e `plot_results.py` (gráficos). |

## 2.2 Visão geral: a vida de um download

1. **Preparação** (`run_test.sh`): gera um arquivo aleatório (`head -c N /dev/urandom`) e roda `mkmeta` para criar o `.meta`.
2. **Início**: sobe o peer A (seeder, já tem o arquivo na pasta) e espera 0,5 s; depois sobe B, C, D com a lista de vizinhos conforme a topologia (mesh ou line).
3. **Cada peer** (`main`, `peer.c:363`) lê o `.meta`, decide se é seeder ou leecher (`storage_init`), abre o socket servidor e dispara **uma thread cliente por vizinho**.
4. **Cada thread cliente** conecta ao vizinho, manda HELLO, pede o BITFIELD e entra no laço: escolhe um bloco raro → GET_BLOCK → recebe BLOCK → confere o SHA-256 do bloco → grava no `.part` → marca `have[idx]=1`.
5. Ao completar o último bloco, `finalize_download` confere o SHA-256 do arquivo inteiro, renomeia `.part` para o nome final e o peer vira seeder (continua respondendo).
6. **Verificação**: o script compara o `sha256sum` e o tamanho de cada cópia com o original e grava a linha em `results.csv`.

## 2.3 Metadados – `mkmeta.c` e `meta.c`

**Onde:** `mkmeta.c:10` (main), `meta.c:7` (`meta_save`), `meta.c:22` (`meta_load`), `meta.c:71` (`meta_block_len`).

**Como:** `mkmeta` lê o arquivo bloco a bloco com `fread`, calcula o SHA-256 de cada bloco (`sha256_buf`) e, ao mesmo tempo, alimenta um contexto SHA-256 do arquivo inteiro (`sha256_update`), sem carregar o arquivo todo na memória. O formato em texto é:

```
P2PMETA 1
filename file_5242880.bin
filesize 5242880
blocksize 4096
numblocks 1280
sha256 <64 hex do arquivo>
blocks
<64 hex do bloco 0>
... uma linha por bloco
```

**Validações em `meta_load`:** cabeçalho `P2PMETA 1`; todos os campos presentes; `blocksize` e `filesize` > 0; `numblocks == ceil(filesize / blocksize)`; nome de arquivo sem `/`; hashes hexadecimais válidos. `meta_block_len` devolve o tamanho real do bloco, que é menor no último bloco quando o tamanho não é múltiplo.

**Por quê:** o `.meta` faz o papel do `.torrent`: sem ele o leecher não tem como saber o que é um bloco válido. Guardar o hash *de cada bloco* permite validar cada pedaço assim que chega, de qualquer vizinho, sem confiar nele. Formato texto atende "desenvolver um formato para o arquivo de metadados" e é fácil de depurar. Recusar `/` no nome evita que um `.meta` malicioso escreva fora da pasta do peer.

### 2.4 Protocolo de mensagens – `protocol.c`

**Onde:** `protocol.h:17` (tipos), `protocol.c:8` (`send_all`), `:22` (`recv_all`), `:37` (`msg_send`), `:47` (`msg_recv`).

| Mensagem | Payload | Para que serve |
|---|---|---|
| HELLO (1) | id do peer (texto) | Identifica quem conectou (aparece nos logs). |
| GET_BITFIELD (2) / BITFIELD (3) | vazio / `numblocks` bytes (1 = tem) | Descobrir quais blocos o vizinho possui. |
| GET_BLOCK (4) | índice (uint32 BE) | Pedir um bloco. |
| BLOCK (5) | índice + dados | Resposta com o conteúdo do bloco. |
| NO_BLOCK (6) | índice | "Não tenho esse bloco (ainda)." |
| BYE (7) | vazio | Encerrar a conexão com educação. |

**Como:** toda mensagem é `[1 byte tipo][4 bytes tamanho em big-endian][payload]`. `msg_send` monta o cabeçalho de 5 bytes com `htonl` e envia cabeçalho e payload. `msg_recv` lê exatamente 5 bytes, converte o tamanho com `ntohl`, recusa tamanho acima de `MAX_PAYLOAD` (64 MiB), aloca o payload com `malloc` e lê o resto (o chamador dá `free`).

**Por quê:** TCP é um *fluxo de bytes* sem fronteiras de mensagem, então é obrigatório ter um cabeçalho com o tamanho. `send_all`/`recv_all` repetem em laço porque `send`/`recv` podem transferir menos bytes do que o pedido e podem ser interrompidos (`EINTR`). `MSG_NOSIGNAL` evita que escrever em um socket fechado mate o processo com SIGPIPE; `MSG_MORE` avisa o kernel que o payload vem em seguida, para juntar cabeçalho e dados em um segmento. Big-endian (ordem de rede) garante que máquinas diferentes entendam o mesmo número. O limite de payload impede que um peer defeituoso faça o outro alocar memória absurda.

### 2.5 SHA-256 e utilidades – `sha256.c`, `util.c`

**Como/por quê:** o SHA-256 foi implementado do zero para o projeto não depender de OpenSSL (basta `gcc` e `make`). A API tem forma incremental (`init/update/final`, usada pelo `mkmeta` para o arquivo inteiro), forma direta para um bloco (`sha256_buf`) e `sha256_file` (usada na remontagem). Como o próprio SHA-256 poderia ter bug, os scripts de teste conferem o resultado com o `sha256sum` do sistema. `now_s()` usa `CLOCK_MONOTONIC`, que não pula quando o relógio do sistema muda e por isso serve para medir tempos.

### 2.6 O peer – `peer.c`

#### a) Estado do peer – `struct Peer` e `Neighbor` (`peer.c:34-67`)

**O que é:** uma única variável global `P` guarda tudo: id, ip, porta, o `Meta`, o descritor do arquivo `.part`, os vetores `have[]` (blocos que eu tenho) e `inflight[]` (blocos que alguém já está pedindo), contadores, o mutex `mu` e um array de `Neighbor` (host, porta e o `bitfield` do vizinho). **Por quê:** um processo = um peer, então estado global é simples; as threads compartilham esse estado protegido por mutex (`P.mu` para blocos; `P.logmu` para o log).

#### b) Armazenamento – `storage_init` (`peer.c:115`)

**Como:** se o arquivo final já existe na pasta do peer com o tamanho esperado, o peer é **seeder**: abre somente leitura, marca todos os blocos como `have` e loga `SEEDER`. Caso contrário é **leecher**: cria `nome.part` e usa `ftruncate` para reservar o tamanho total, loga `LEECHER 0/N`. **Por quê:** assim não há flag "eu sou seeder": o que define o papel é o estado em disco, o que também permite retomar. Reservar o tamanho permite gravar qualquer bloco em qualquer posição (`pwrite`), já que os blocos chegam fora de ordem (rarest-first). Ponto de atenção: o seeder confia no arquivo que está na pasta (não revalida o hash na partida).

#### c) Servidor – `make_listener`, `server_thread`, `conn_thread` (`peer.c:198-226, 143-196`)

**Como:** `make_listener` cria o socket TCP, liga a `ip:porta` com `SO_REUSEADDR` e faz `listen`. `server_thread` espera conexões com `poll(..., 200ms)` + `accept`; para cada conexão cria uma **thread desanexada** (`conn_thread`) que fica em laço lendo mensagens:

- **HELLO** → registra quem é (log "conexao recebida de X").
- **GET_BITFIELD** → copia `have[]` sob o mutex e envia fora do mutex.
- **GET_BLOCK** → sob o mutex confere `have[idx]` e lê o bloco com `pread`; se tem, envia BLOCK e soma em `served_blocks/bytes`; se não tem, envia NO_BLOCK.
- **BYE** ou conexão fechada → encerra a thread e fecha o socket.

**Por quê:** o `poll` com timeout deixa a thread notar `P.stop` (Ctrl-C/SIGTERM) sem ficar presa no `accept`. Thread por conexão é o jeito mais simples de atender vários vizinhos ao mesmo tempo (concorrência exigida). `TCP_NODELAY` desliga o algoritmo de Nagle: como o protocolo é pedido/resposta com mensagens pequenas, Nagle + ACK atrasado criaria atrasos de dezenas de milissegundos. O servidor lê `have[]` ao vivo, por isso **um bloco recém-gravado já é servido a outros** – isso é o "seeder em cascata" do enunciado.

#### d) Cliente – `connect_to`, `client_worker` (`peer.c:229-245, 311-341`)

**Como:** uma thread por vizinho. `connect_to` tenta conectar a cada 250 ms até conseguir (ou até `stop`); ao conectar liga `TCP_NODELAY` e `SO_RCVTIMEO` de 15 s. Depois: HELLO → `refresh_bitfield` (GET_BITFIELD/BITFIELD) → laço: `pick_block` → `request_block`. Se não há bloco para pedir desse vizinho, dorme 100 ms e relê o bitfield dele. A cada 256 blocos recebidos relê o bitfield de novo. Se a conexão falha, libera o bloco em `inflight`, loga `WARN` e reconecta. Ao terminar manda BYE.

**Por quê:** o repetir da conexão resolve a ordem de inicialização (os peers sobem em momentos diferentes e a lista de vizinhos é estática). O timeout de recebimento impede que uma thread fique travada para sempre se o vizinho parar de responder. O bitfield é relido periodicamente porque o protocolo **não tem mensagem HAVE**: é assim que o peer descobre os blocos que o vizinho foi ganhando (modelo pull). Uma thread por vizinho permite baixar de vários peers em paralelo.

#### e) Escolha do bloco – `pick_block` (rarest-first) (`peer.c:260`)

**Como:** sob o mutex, percorre os blocos `0..N-1` e ignora os que já tenho (`have`), os já pedidos (`inflight`) e os que *esse vizinho* não tem. Para cada candidato conta em quantos vizinhos o bloco existe (*raridade*) e fica com o de menor contagem (para cedo se a raridade é ≤ 1). O escolhido é marcado em `inflight`, para que outra thread cliente não peça o mesmo bloco ao mesmo tempo.

**Por quê:** rarest-first é a estratégia do BitTorrent: baixar primeiro o que poucos têm mantém o enxame saudável, porque se o único peer que tinha um bloco raro sair, o bloco some. `inflight[]` evita pedidos duplicados entre as threads. **Custo:** a varredura é linear e recomeça do bloco 0, passando pelos que já tenho; com 20 480 blocos (1 KB) isso pesa mais que com 5 120 (4 KB), e roda dentro do mutex global (um dos prováveis contribuintes da lentidão com bloco pequeno e mais peers – hipótese baseada na leitura do código, não medida).

### f) Receber, validar e gravar – `request_block`, `store_block` (`peer.c:289, 274`)

**Como:** `request_block` envia GET_BLOCK e espera a resposta (um pedido por vez por conexão). Se vier NO_BLOCK, limpa o bit daquele bloco no bitfield do vizinho e libera o `inflight`. Se vier BLOCK, confere índice e tamanho esperado (`meta_block_len`), calcula o SHA-256 e compara com `meta.block_hash[idx]`. **Hash errado:** descarta, para de pedir esse bloco àquele vizinho e libera para pedir a outro. **Hash certo:** `store_block` grava com `pwrite` no offset `idx × blocksize` (sem lock), e depois, sob o mutex, marca `have[idx]`, atualiza contadores por vizinho (`blocks_recv`, `bytes_recv`, usados no relatório de origem) e zera `inflight`. O thread que grava o último bloco chama `finalize_download`.

**Por quê:** validar antes de gravar garante que bloco corrompido ou falso nunca entra no arquivo (exigência "após todos os blocos serem recebidos e verificados"). `pwrite`/`pread` usam offset explícito, então várias threads escrevem em posições diferentes do mesmo arquivo sem disputar um ponteiro de arquivo compartilhado.

### g) Remontagem – `finalize_download` (`peer.c:91`)

**Como:** `fsync` do `.part`; calcula o SHA-256 do arquivo inteiro (`sha256_file`) e compara com o do `.meta`. Se confere: `rename(.part → nome final)`, marca `verified`, loga `DOWNLOAD COMPLETO … tempo, taxa, sha256=OK` e lista a origem dos blocos (`origem host:porta -> N blocos (x%)`). Se não confere, loga erro e não renomeia. **Por quê:** o `.part` deixa claro que o arquivo ainda é parcial; o `rename` é atômico, então o nome final só existe com conteúdo verificado. A lista de origem é a evidência pedida pelo enunciado ("confirmar nos logs que os blocos foram recebidos das fontes esperadas").

### h) `main`, sinais e vida do processo (`peer.c:344–427`)

**Como:** `getopt_long` lê `--id --ip --port --meta --dir --neighbors --log --linger --quiet`. `SIGPIPE` é ignorado e `SIGINT/SIGTERM` só setam `P.stop`. Ordem: carregar `.meta` → alocar bitfields dos vizinhos → `storage_init` → abrir servidor → disparar threads cliente (somente se não for seeder) → laço que a cada 100 ms verifica se completou. Quando `verified`, loga "agora atuando como SEEDER". Por padrão (`--linger -1`) o peer continua rodando e servindo indefinidamente; com `--linger N` sai N segundos depois de completar. No fim loga quantos blocos serviu. **Por quê:** quem terminou precisa continuar no ar para servir os que ainda baixam; o script de teste é quem mata os peers. Tratar sinais com flag (em vez de `exit`) deixa as threads encerrarem sozinhas.

### 2.7 Concorrência: o que protege o quê

| Recurso | Proteção | Observação |
|---|---|---|
| `have[]`, `inflight[]`, `have_count`, bitfields dos vizinhos, contadores de serviço | mutex `P.mu` | Seções críticas curtas; o `pread` do servidor fica dentro do lock para ser atômico com o teste `have[idx]`. |
| Arquivo `.part` | nenhuma (offset explícito) | `pwrite` / `pread` não usam o ponteiro de arquivo; blocos distintos = regiões distintas. |
| Saída de log | mutex `P.logmu` | Evita linhas misturadas entre threads. |
| `stop`, `complete`, `verified` | `volatile` (`sig_atomic_t` para `stop`) | Flags simples lidas por várias threads. |

### 2.8 Scripts de teste – `run_test.sh`

**Como:** recebe nº de peers, tamanho do bloco, tamanho do arquivo, topologia e rótulo. Reaproveita o arquivo aleatório entre testes (`data/file_N.bin`). A função `neighbors_of` monta a lista de vizinhos: em **mesh** cada peer lista todos os outros; em **line** só o anterior e o seguinte (A–B–C–D). Sobe os peers (A primeiro, com `sleep 0.5`), espera até todos os leechers logarem `DOWNLOAD COMPLETO` (timeout de 300 s → `TIMEOUT`), mata os peers e verifica `sha256sum` e `stat -c%s` de cada cópia. Extrai o maior tempo de download dos logs e grava uma linha no CSV (`tempo_total_s` = relógio do script; `maior_tempo_peer_s` = tempo do peer mais lento medido pelo próprio peer). **Por quê:** automatiza exatamente os passos de "Para cada teste, seguir" do enunciado (preparação, início, monitoramento, verificação) e deixa os dados prontos para a média e os gráficos.

### 2.9 Requisito do enunciado → onde está no código

| Requisito | Implementação |
|---|---|
| Simetria servidor + cliente | `server_thread` / `conn_thread` + `client_worker` no mesmo processo (`peer.c:208, 143, 311`) |
| Configuração com IP, porta e vizinhos | `--ip --port --neighbors` + `parse_neighbors` (`peer.c:351`); sem tracker |
| Fragmentação em blocos fixos | `mkmeta.c` (blocos de `blocksize`; último pode ser menor) |
| Leecher conecta a todos os vizinhos e pede blocos até completar | uma thread `client_worker` por vizinho; laço enquanto `!P.complete` |
| Virar seeder ao receber blocos | `GET_BLOCK` do servidor consulta `have[]` ao vivo; log "agora atuando como SEEDER" |
| Socket com concorrência | threads POSIX + `poll` no accept + `SO_RCVTIMEO` (o "não-bloqueio" é tratado com threads e timeouts, não com I/O assíncrono) |
| Serialização / protocolo | `.meta` em texto; mensagens `[tipo][tam][payload]` (`protocol.c`) |
| Registro de blocos | `have[]`, `inflight[]`, bitfield por vizinho |
| Montagem e verificação | hash por bloco (`request_block`) + hash final e `rename` (`finalize_download`) |
| Logs com a origem dos blocos | `blocks_recv` / `bytes_recv` por vizinho, impressos em `finalize_download` |

### 2.10 Pontos fracos conhecidos (assuma na apresentação)

- **Um pedido por vez por conexão** (sem pipelining): a velocidade fica limitada pela latência por bloco (~0,6–0,8 ms), e não pela largura de banda. Melhoria: manter vários GET_BLOCK em andamento por conexão.
- **Sem mensagem HAVE**: o peer só descobre blocos novos relendo o bitfield; por isso os leechers trocam pouco entre si. Melhoria: o peer avisar os vizinhos a cada bloco novo.
- **Mutex global + varredura linear no `pick_block`**: contenção com bloco pequeno e mais threads. Melhoria: ponteiro para o próximo candidato, contadores de raridade incrementais.
- **Outliers intermitentes**: no 1º benchmark, execuções na topologia line chegaram a 105 s (10 KB); no 2º não ocorreram em 180 execuções. Causa não confirmada; pista: múltiplo do timeout de 15 s do cliente.
- Thread por conexão e `MAX_NEIGH = 16` limitam a escala; é suficiente para os testes (2–4 peers).
- Teste só em localhost: não mede rede real; o ambiente (WSL2) adiciona jitter.
- O seeder confia no arquivo local (não revalida o hash ao iniciar); os gráficos não mostram as barras de erro.

### 2.11 Perguntas prováveis e respostas curtas

1. **Por que threads e não `select` / `epoll`?** Thread por conexão é mais simples e atende o requisito de concorrência; o custo é escalabilidade, aceitável para poucos peers.
2. **Como evita dois pedidos do mesmo bloco?** `inflight[]` marcado sob o mutex em `pick_block`.
3. **Se um vizinho cair?** A thread cliente detecta a falha (erro ou timeout de 15 s), libera o `inflight` e reconecta; os blocos que ele tinha passam a ser pedidos de outros vizinhos.
4. **E se chegar um bloco corrompido?** O SHA-256 do bloco não bate com o `.meta`; ele é descartado e o bit daquele vizinho é limpo.
5. **Como garante que o arquivo final está completo e correto?** Escreve em `.part`, confere o SHA-256 do arquivo inteiro e só então faz `rename`; o script ainda recalcula com `sha256sum` e confere o tamanho.
6. **Por que 4 KB é mais rápido que 1 KB?** Há 4× menos blocos e cada bloco custa ~0,6–0,8 ms de ida e volta; os tempos medidos seguem essa proporção (razões de 2,9× a 3,7×).
7. **Por que mais peers não acelerou?** O seeder serve ~95% dos blocos, tudo roda na mesma máquina (CPU/mutex) e os leechers descobrem blocos uns dos outros só quando relêem o bitfield; no 2º benchmark ficou até mais lento (11,8 → 20,3 s com 1 KB).
8. **Por que não usar tracker?** O enunciado permite lista estática de vizinhos para focar no protocolo P2P.

Números: média de 5 rodadas do 2º benchmark (`results/results_avg.csv`, coluna do peer mais lento). Referências de linha em `src/peer.c` conforme o código atual do repositório.